

Boa noite, pdsilva! Excelente questão. Quando você tira a memória de vídeo de dentro do microcontrolador (ou do circuito principal de controle) e a coloca no barramento externo, a arquitetura clássica dos anos 80 e 90 para placas de vídeo de texto é exatamente a resposta para evitar esse "gargalo" de processamento.

Para fazer isso de forma puramente em hardware (sem que o Pico precise ler o ASCII, traduzir e gerar o pixel), você precisa de um circuito clássico de **geração de caracteres por hardware**.

A lógica consiste em transformar o processo em um "pipeline" sequencial de hardware usando três componentes principais externos:

1. **VRAM (Memória de Vídeo Externa):** Armazena apenas os códigos ASCII (e opcionalmente atributos de cor).
2. **ROM de Caracteres (Font ROM):** Armazena os bitmaps (as matrizes de pixels) de cada caractere.
3. **Shift Register (Registrador de Deslocamento):** Transforma os bytes paralelos da ROM em um fluxo serial de pixels (os pontos na tela).

Aqui está como esse circuito se organiza e funciona na prática:

O Fluxo do Hardware (Passo a Passo)

1. **Os Contadores de Vídeo (Geradores de Endereço):** O seu circuito gerador de sincronismo (H-Sync/V-Sync), que pode ser lógica discreta ou o próprio Pico gerando os tempos, mantém contadores de pixels, linhas e colunas.
2. **Acessando a VRAM (O código ASCII):**
A partir da linha e da coluna atuais da tela, os contadores geram o endereço da **VRAM**. A VRAM devolve o código ASCII correspondente àquela posição de texto (por exemplo, `0x41` para a letra 'A').
3. **Acessando a ROM de Caracteres (O Bitmap):**
O código ASCII vindo da VRAM não vai para a CPU; ele vai direto para as **linhas de endereço superiores** da ROM de Caracteres. Enquanto isso, o contador de "linhas do caractere" (que conta de 0 a 7 ou 0 a 15, dependendo do tamanho da fonte) é injetado nas **linhas de endereço inferiores** da ROM.
* *Resultado:* A ROM recebe (Código ASCII + Linha Atual) e entrega na saída um byte que representa exatamente os pixels daquela linha específica daquela letra.

4. **Serialização (O Shift Register):**

Esse byte da ROM é carregado em paralelo em um Shift Register de alta velocidade (como o clássico 74HC165). O clock de pixel (*dot clock*) vai deslocando esses bits para fora, um a um. Cada bit `1` vira pixel aceso (cor de frente) e cada `0` vira pixel apagado (fundo), que seguem para o DAC.

Como estruturar isso sem desperdiçar ciclos do Pico?

Se você quiser implementar isso com chips discretos (ou dentro de uma CPLD/FPGA para economizar espaço no seu barramento modular), a arquitetura de acesso à VRAM geralmente segue dois caminhos:

1. Acesso Compartilhado (Dual-Port RAM ou Multiplexada)

Para a CPU (ou o Pico) poder escrever o texto na VRAM sem atrapalhar o gerador de vídeo, usa-se uma memória RAM de porta dupla (**Dual-Port RAM**), onde o Pico escreve por um lado e o circuito de vídeo lê pelo outro.

* *Alternativa barata:* Multiplexadores de barramento (como o 74HC257). Quando o sinal de vídeo está na área visível, os multiplexadores entregam o controle dos endereços da VRAM para os contadores de vídeo. Durante o *Blanking* (H-Blank ou V-Blank), o controle é devolvido para o Pico escrever os novos códigos ASCII.

2. O Pico apenas como "Gerador de Endereços e Sincronismo"

Se você não quiser usar chips de lógica para os contadores de pixel/linha, o Pico pode usar as fiações de PIO apenas para "apontar" para a VRAM externa e gerar o HSync/VSync. Ele mudaria o endereço na VRAM a cada caractere, e o hardware externo (ROM + Shift Register) cuidaria do resto do trabalho pesado de abrir a matriz de pontos e disparar o DAC.

Resumo dos Chips Clássicos para o Circuito

Se você fosse montar isso hoje com lógica discreta na bancada, o "core" do circuito usaria:

- * **SRAM comum (ex: 62256):** Como VRAM para guardar os blocos de texto.
- * **EEPROM/Flash (ex: 28C64 ou 29F010):** Gravada com os bitmaps da sua fonte (ex: matriz 8x8 ou 8x16 do IBM PC).
- * **74HC165:** Como Shift Register para serializar os pixels para o DAC.
- * **74HC157 / 74HC257:** Multiplexadores para isolar o barramento do Pico do barramento do circuito de vídeo.

Essa era a mágica por trás de chips dedicados como o **Motorola 6845** (usado no IBM MDA/CGA e no BBC Micro). Ele não processava pixels; ele apenas gerenciava os endereços da VRAM e os tempos de sincronismo, deixando a geração de pontos para a Font ROM e o Shift Register externos.

Essa lógica faz sentido para o que você está visualizando para o barramento externo? Se quiser, podemos detalhar como ficaria a pinagem ou a multiplexação dessa VRAM.

Perfeito, pdsilva! Essa é exatamente a "virada de chave" para entender a lógica de texto por hardware. O segredo está em como o circuito de vídeo enxerga a tela.

A tela VGA faz a varredura linha por linha de pixels (horizontalmente), de cima para baixo. Como uma fonte 8x16 tem **16 linhas de pixel de altura**,

o feixe de elétrons (ou o sinal digital) vai demorar **16 linhas horizontais da tela VGA** para desenhar completamente uma única linha de texto.

Para o caractere 'A' na coluna 0, linha de texto 5, o circuito precisa fatiar o desenho dele ao longo dessas 16 varreduras da tela. Vamos ver como os contadores de hardware fazem isso:

1. O Cálculo das Coordenadas (Divisão de Tarefas)

O seu gerador de vídeo (sejam contadores discretos ou o Pico) possui dois contadores principais para a vertical:

- **Contador de Linhas de Texto (Y_Texto):** Indica em qual linha de blocos de texto a tela está (no seu exemplo, **5**).
- **Contador de Linhas do Caractere (Y_Pixel):** Conta de **0 a 15**, indicando qual "fatia" horizontal da fonte está sendo desenhada naquele exato momento.

2. O Passo a Passo da Varredura (Linha por Linha)

Quando o feixe VGA chega na linha de texto 5, o contador `Y_Pixel` zera e começa a contagem daquela linha de texto:

Tela VGA: Varredura Física nº 80 (Exemplo: Linha de Texto 5, Y_Pixel = 0)

1. O circuito calcula o endereço da VRAM para a coluna 0, linha 5. A VRAM responde com `0x41` ('A').
2. O circuito joga `0x41` nos bits superiores de endereço da **ROM de Fonte**, e joga `0` (porque `Y_Pixel = 0`) nos bits inferiores.
3. A ROM de Fonte olha para o mapa do caractere `0x41`, na sua primeiríssima linha (linha 0), e entrega o byte correspondente (geralmente `0x00` ou `0x18`, dependendo do design do 'A', já que o topo costuma ser vazio ou apenas a pontinha da letra).
4. Esse byte vai para o Shift Register e é cuspidos na tela.

Tela VGA: Varredura Física nº 81 (Ainda na Linha de Texto 5, mas agora Y_Pixel = 1)

A tela terminou a linha anterior, deu o H-Sync, e agora está desenhando a linha de pixel logo abaixo.

1. O feixe passa de novo pela coluna 0. O circuito lê a VRAM na mesma posição (linha 5, coluna 0) e obtém o mesmo `0x41`.

2. Mas agora, o contador Y_Pixel mudou para **1**.
3. O endereço da ROM de Fonte agora é $0x41$ nos bits superiores e 1 nos bits inferiores.
4. A ROM agora entrega a **segunda linha de pixels** da matriz do 'A' (algo como $0x3C$, que representa o topo da letra `..XXXX..`).
5. O Shift Register recebe $0x3C$ e desenha essa fatia na tela.

Tela VGA: Varredura Física nº 85 ($Y_Pixel = 5$ - O meio do 'A')

Algumas linhas depois, o feixe está passando pelo meio do caractere.

1. VRAM continua entregando $0x41$.
2. A ROM recebe $0x41 + Y_Pixel = 5$.
3. Nessa altura, o 'A' tem aquela barra horizontal do meio. A ROM vai entregar um byte cheio de bits 1 (ex: $0x7E$, que em binário é `.XXXXXX.`).
4. O Shift Register serializa isso e você vê o traço horizontal do 'A' na tela.
Como a ROM de Fonte se organiza na memória?

Para que isso funcione direto no barramento sem lógica complexa, a ROM de Fonte é gravada pensando exatamente nessa matemática. Cada caractere ocupa **16 bytes seguidos** na memória:

- Endereço $0x410$ (letra 'A', linha 0) $\rightarrow$ Byte da linha 0 do 'A'
- Endereço $0x411$ (letra 'A', linha 1) $\rightarrow$ Byte da linha 1 do 'A'
- ...
- Endereço $0x41F$ (letra 'A', linha 15) $\rightarrow$ Byte da linha 15 do 'A'

A Mágica do Barramento:

Se você juntar os fios do barramento de endereço da ROM de Fonte assim:

- **A11 até A4:** Conectados direto na saída de dados da VRAM (os 8 bits do ASCII $0x41$).
- **A3 até A0:** Conectados direto no seu contador Y_Pixel (que conta de 0 a 15).

O próprio hardware faz a soma de ponteiros instantaneamente! Sempre que o Y_Pixel incrementa a cada nova linha da tela, a ROM aponta automaticamente para a próxima fatia do caractere, sem que o Pico precise mover um único bit para isso.

Ficou mais claro como o circuito "fatia" a letra à medida que a varredura da tela vai descendo?

Para desenhar a palavra **"pdsilva"** na tela, o circuito de vídeo vai processar essa linha de texto horizontalmente, caractere por caractere, da esquerda para a direita.

Como a fonte tem 16 linhas de altura, a tela VGA vai passar por essa mesma linha de texto **16 vezes** (uma para cada linha de pixel/varredura). Considerando a **primeira varredura física** (Linha do Caractere = 0, o topo das letras), o desenho de 1 linha de texto acontece de forma sucinta assim:

O Fluxo da Esquerda para a Direita (Varredura Horizontal):

1. **Letra "p" (Coluna 0):** O circuito lê a VRAM $\rightarrow$ descobre o ASCII do p $\rightarrow$ combina com a Linha 0 na ROM de Fonte $\rightarrow$ O Shift Register recebe o byte e solta os 8 pixels do topo do p (provavelmente tudo 0, vazio).
2. **Letra "d" (Coluna 1):** O contador de colunas avança. A VRAM entrega o ASCII do d $\rightarrow$ ROM de Fonte entrega a Linha 0 do d $\rightarrow$ Shift Register solta os 8 pixels na tela (a ponta da haste vertical do d).
3. **Letra "s" (Coluna 2):** A VRAM entrega o ASCII do s $\rightarrow$ ROM entrega a Linha 0 do s $\rightarrow$ Shift Register solta os 8 pixels (o topo da curva do s).
4. **Letra "i" (Coluna 3):** A VRAM entrega o ASCII do i $\rightarrow$ ROM entrega a Linha 0 do i $\rightarrow$ Shift Register solta os 8 pixels (o pingo do i).
5. **Letra "l" (Coluna 4):** A VRAM entrega o ASCII do l $\rightarrow$ ROM entrega a Linha 0 do l $\rightarrow$ Shift Register solta os 8 pixels (o topo da haste do l).

6. **Letra "v" (Coluna 5):** A VRAM entrega o ASCII do `v` $\rightarrow$ ROM entrega a Linha 0 do `v` $\rightarrow$ Shift Register solta os 8 pixels (vazio, o topo do `v` começa mais abaixo).
7. **Letra "a" (Coluna 6):** A VRAM entrega o ASCII do `a` $\rightarrow$ ROM entrega a Linha 0 do `a` $\rightarrow$ Shift Register solta os 8 pixels (vazio).

O Resultado

Ao final dessa primeira passagem de 56 pixels de largura (7 letras $\times$ 8 pixels), o circuito gera o sinal de **H-Sync** (retorno horizontal). Na próxima linha da tela VGA, o contador da ROM muda para a **Linha do Caractere = 1**. O circuito lê exatamente os mesmos códigos ASCII na RAM (`p`, `d`, `s`, `i`, `l`, `v`, `a`), mas a ROM de Fonte agora vai entregar a segunda fatia de pixels de cada letra, colando-as perfeitamente logo abaixo da primeira passagem. Esse ciclo se repete até a linha 15, completando o nome **pdsilva** na tela.

Excelente pergunta! Você chegou no coração do design de hardware de vídeo. A relação entre o contador da RAM (VRAM) e o contador da ROM de Fonte é uma relação de **velocidade (frequência) e de divisão de escala**.

Para entender como eles se conectam, pense em como o feixe caminha na tela: ele anda muito rápido na horizontal (pixels) e mais devagar na vertical (linhas).

A relação exata entre os dois contadores funciona da seguinte forma:

1. O Contador de Endereços da RAM (VRAM)

O endereço da VRAM determina **qual caractere** está sendo lido.

- **Na Horizontal:** Ele precisa mudar a cada **8 pixels** (já que a largura do caractere é 8). Ou seja, o clock que faz a VRAM avançar para a próxima coluna é o seu $\text{Pixel_Clock} / 8$ (geralmente chamado de *Character Clock*).
- **Na Vertical:** Quando a linha horizontal da tela termina (H-Sync), o contador da VRAM **não deve avançar para a próxima linha de texto**

ainda. Ele precisa voltar para o início da mesma linha de caracteres, porque a tela ainda precisa desenhar as outras 15 fatias daquelas mesmas letras. Ele só vai para a próxima linha de texto da VRAM após a ROM de fonte terminar todas as suas 16 linhas.

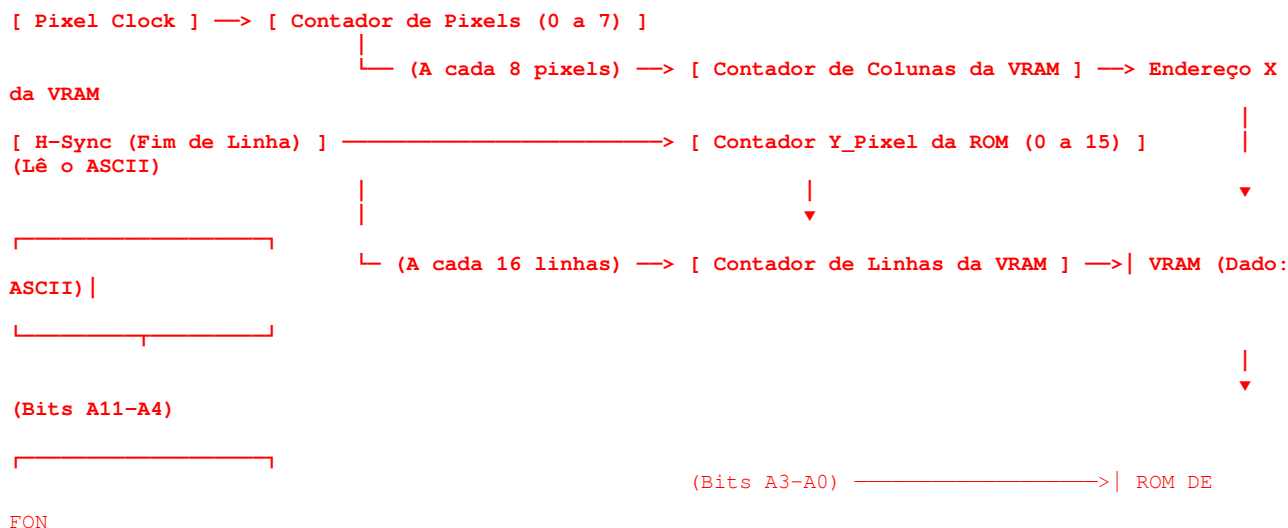
2. O Contador de Endereços da ROM de Fonte

O endereço da ROM é composto por duas partes combinadas no barramento, e cada uma ouve um contador diferente:

- **A parte alta do endereço (ASCII):** Vem direto dos dados de saída da VRAM. Ela muda na velocidade do *Character Clock* (horizontal).
- **A parte baixa do endereço (Linha do Caractere):** Esta parte ouve o **Contador de Linhas Verticais** da tela. Ela só incrementa **1 vez a cada H-Sync** (fim de linha física da tela). Ela conta de 0 a 15 e, quando reseta para 0, envia um sinal para o contador da VRAM finalmente avançar para a próxima linha de texto.

A Arquitetura de Conexão (Como os fios se ligam)

Para visualizar isso em hardware discreto (ou lógica PIO/CPLD), imagine os contadores divididos em blocos:



Resumo da Relação de Tempo:

- **Contador da ROM (Linhas 0-15):** É um contador síncrono com o **H-Sync** da tela. Ele muda devagar (uma vez por linha horizontal da tela).

- **Contador da RAM (Colunas):** É um contador síncrono com o **Pixel Clock / 8**. Ele corre muito rápido na horizontal, reinicia a cada linha da tela, e só muda seu "offset" vertical quando o contador da ROM (0-15) transborda (reseta).

Em circuitos clássicos, para evitar ter que ficar fazendo contas complexas de multiplicação (como $\text{Linha} * 80 + \text{Coluna}$), os projetistas usavam contadores binários simples e faziam a VRAM ter tamanhos baseados em potências de 2 (como 64 ou 128 colunas por linha, mesmo que a tela só mostrasse 80), conectando os bits dos contadores diretamente nos fios de endereço da RAM.